Structures, Unions and Enumerations

Ilex Travis
ENGR-2304-00003
Roderick Crowder
10/5/2026

Structures : Purpose

In C programming the core purposes and benefits are plenty.

The first of them is, the ability to efficiently group different data types. Distinct from arrays which only hold elements of the same data type. A structure can bring together integers, floats, characters, arrays, and pointers under one name.

The next is modelling real-world entities which is a key concept. They make it straightforward to represent complex objects as a single record. For example, a student (name, ID, grade) or an employee (name, salary, ID) can each be stored as one unified unit rather than as scattered variables.

The third is systematic ability to pass data as a unit sort of like an array but with more precisely located related data. You can move an entire collection of data around together. The set can then be passed into a function or returned from one easily, which removes the strain of managing dozens of individual loose variables.

The fourth is that structures in C programming build advanced data structures. They form the foundational building blocks for more sophisticated constructs like linked lists, trees, and graphs. This is where each node is typically a structure that holds data along with pointers.

The fifth core purpose and benefit is, by grouping logically connected data structures improve code organization. They keep code clean, readable, and easier to maintain.

Structures allow a real-world entity to be represented by merging several data types into a single, cohesive unit. This keeps programs well organized, easy to read, and simple to expand as they grow.

```
struct  student  {
   char  name{50};
   int  age;
   float  gpa;
};

//  Declaring and accessing a structure
variable
struct  student  s1;
S1.age  =  20;
```

Structures : Syntax

https://en.cppreference.com/c/language/struct

Syntax to Define a Structure in C

```
struct structName
{
   // structure definition
   Data_type1 member_name1;
   Data_type2 member_name2;
   Data_type3 member_name3;
};
```

Definition of the Syntax

Keyword struct: A struct is a type consisting of a sequence of members whose storage is allocated in an ordered sequence (as opposed to union, which is a type consisting of a sequence of members whose storage overlaps.

structName: A unique identifier that names the structure template so it can be referenced later to declare structure variables.

data_Type: The data type indicates the type of the data members of the structure. A structure can have data members of different data types.

member_name: A unique variable name inside the structure that allows access to that specific field using the dot operator.

Structures : Benefits

```c
#include <stdio.h>

struct Point {
    int x;
    int y;
};

int main() {
    struct Point p = {5, 10};
    printf("Coordinates: %d, %d\n", p.x, p.y);
    return 0;
}
```

Structures in C offer a powerful means of organizing and handling related data with efficiency. They sharpen clarity, reduce complexity, and are indispensable for constructing larger, well-structured programs. By unifying related variables under one name, they give C programs a more modular, maintainable, and expressive design.

Key Benefits of Using Structures in C

Structures keep related data organized in one place. Groups related data into a single unit, making it more manipulable and easier to manage complex records (e.g., student, employee, product).

Structures make code easier to read and maintain. Meaningful structure names and grouped fields make code more self-documenting and easier to maintain. Keeping track of data becomes effortless.

Structures let you model real-world entities directly. Structures can directly represent entities in the real world, such as a Book with title, pages, and price. This allows you to hold real things, in data storage spaces.

Structures simplify how you handle data. Instead of managing multiple separate variables, you can work with a single structure variable containing all related data. This cuts back directory issues, direct access to the data you need.

Structures make passing data to functions cleaner. All related data can be handed to a function through a single parameter, which eliminates the need for a long list of separate arguments. Reliqueshes the need for complex arrays of unrelated information.

Structures serve as the foundation for advanced data structures. Structures are basis for linked lists, trees, and other advanced data structures.

Structures can be reused across your code. Define a structure once and reuse it across multiple files or programs. Making them extremely dynamic across projects.

Structures help prevent misuse of related data. A structure ties together the data items it contains, which discourages misuse by keeping each field’s type and purpose clearly defined. Avoids simple mistakes by keeping things organized and named properly.

Unions : Purpose

The main purpose of a union in C programming is to save memory by allowing multiple variables to share the exact same block of memory.

All members of a union share one memory location. They all begin at the same address, so assigning a value to one member overwrites the values of the others. A union is only as large as its biggest member. A struct allocates separate space for every field, but a union reserves only enough room for its largest data member.

Only one member holds a valid value at a time. Since the members overlap in memory, just the most recently written member contains meaningful data.

```
struct MyStruct {      union MyUnion {
    int i;   // 4 bytes    int i;   // 4 bytes
    char c;  // 1 byte     char c;  // 1 byte
};                     };
// Total size: 5+ bytes  // Total size: 4 bytes
```

Common Uses

Unions conserve memory in low-memory environments. They are ideal for embedded systems, where only one of several possible values (such as an ID number or a text string) needs to be stored at a given moment.

Unions make type punning straightforward. You can reinterpret the same memory representation as a different data type, such as reading the raw bytes of a float as an integer.

Unions simplify hardware register manipulation. They map hardware registers or bit-fields into different formats or byte layouts through one unified view.

Unions form the core of tagged (variant) structures. A union is often embedded inside a standard struct along with an enumeration tag that keeps track of which member is currently active.

Unions : Syntax

Defined by the programmer, a union brings together different data types in one memory location. Rather than receiving separate space as they would in a struct, all union members share a single block of memory. Size is set by the largest member, so the union reserves only enough room for that one.

Here is a summary of the syntax for defining, declaring, and using unions in C.

```
union union_name {
    data_type1 member1;
    data_type2 member2;
    // More members can be added
};
```

Where

- union_name: The name of the union.
- data_type: The type of each member (e.g., int, float, char[], etc.).
- member: The names of the individual members that share the same memory space.

Example of Union Declaration:

```
union Data {
 int i;
 float f;
 char str[20];
};
```

Unions : Benefits

Sharing one block of memory is what defines a union. Every member occupies the exact same location, which sets it apart from a struct.

Memory conservation comes from the union's compact size. Its total size equals that of its largest member alone, which drastically reduces RAM usage in resource-constrained environments like embedded systems. Through type punning, the same raw bytes can be viewed as different data types. A 32-bit integer, for example, can be read back as four individual 8-bit bytes, which suits RGBA color values and IP addresses. Hardware register mapping is simpler with unions. In device-driver programming, a register can be addressed as a whole word or manipulated through its individual bit-fields. Variant data and protocol parsing benefit from a nested union. When a packet or file format carries several message types but only one is active at a time, a union inside a struct packages the payload cleanly.

```
#include <stdio.h>

union Data {
   int i;
   float f;
};

int main() {
   union Data d;

   d.i = 10;
   printf("Int: %d\n", d.i);

   d.f = 5.5; // Overwrites the integer
   printf("Float: %.1f\n", d.f);

   return 0;
}
```

Enumerations : Purpose

An enumeration (enum) in C is a user-defined data type that assigns human-readable names to a set of integer constants.

Main Purpose and Benefits

- Improved Readability: Obscure “magic numbers” such as 0, 1, or 2 are replaced with descriptive identifiers like RED, YELLOW, and GREEN, which makes the code self-documenting.
- A Better Alternative to #define Macros: Where preprocessor macros are simple text substitutions, enum values follow normal C scoping rules and are grouped under a specific type name. The compiler also generates sequential values automatically.
- Control Flow Integration: Enums work seamlessly inside switch-case statements and if-else checks, which keeps the handling of states, error codes, menus, and flags clean and straightforward.
- Maintainability: Constant values can be inserted, reordered, or updated in one central definition without breaking hardcoded numeric logic elsewhere in the codebase.

```c
#include <stdio.h>

// Define the enum
enum TrafficLight {
    RED,    // Assigned 0 automatically
    YELLOW, // Assigned 1 automatically
    GREEN   // Assigned 2 automatically
};

int main() {
    enum TrafficLight current = RED;

    if (current == RED) {
        printf("Stop! Value is %d\n", current); // Prints 0
    }
    return 0;
}
```

Enumerations : Syntax

An enum is defined by using the ‘enum’ keyword in C, and the use of a comma separates the constants within. The basic syntax of defining an enum is:

```
enum enum_name{int_const1, int_const2, int_const3, …. int_constN};
```

In the above syntax, the default value of int_const1 is 0, int_const2 is 1, int_const3 is 2, and so on. However, you can also change these default values while declaring the enum. Below is an example of an enum named cars and how you can change the default values.

```
enum cars{BMW, Ferrari, Jeep, Mercedes-Benz};
```

Here, the default values for the constants are:

BMW=0, Ferrari=1, Jeep=2, and Mercedes-Benz=3. However, to change the default values, you can define the enum as follows:

```
enum cars{

BMW=3,

Ferrari=5,

Jeep=0,

Mercedes-Benz=1

};
```

Enumerations : Benefits

Enumerations (enum) in C replace raw numeric constants with descriptive, human-readable names.

Key Benefits

- Improved Readability: Code becomes self-documenting when names like GREEN or PENDING stand in for cryptic numbers such as 2 or 1.
- Automatic Numbering: C assigns sequential integer values to enum members, beginning at 0, unless you specify custom values.
- Better Debugging: Debuggers such as GDB can show the actual name of an enumeration constant instead of a raw integer during inspection.
- An Alternative to Preprocessor Macros: Where #define directives are plain text substitutions, enums respect normal scope rules and group related constants cleanly under a shared tag.
- Control Flow Integration: Because enums evaluate to integers, they fit in cleanly and expressively inside switch statements and if-else branches.
- Easy Maintenance: Changing or reordering values in a single enum definition carries the update through the program logic.

```c
#include <stdio.h>

enum Status {
    LOW = 10,
    MEDIUM,
    HIGH = 20
};

int main(void) {
    enum Status battery = MEDIUM;

    if (battery == MEDIUM) {
        printf("Battery Alert level: %d\n", battery);
    }

    return 0;
}
```

Major Differences

Major Differences

The first of them is the function. A function is a named, reusable block of code that performs a task. It takes in data through parameters and can hand a value back when it finishes. It is the part of a program that does something.
Example: int max(int a, int b);

The next is the array. An array stores many values under one name, but every item has to be the same data type. Its size is fixed once it is declared, and each item is reached through an index that begins at 0. It is the part of a program that holds a list of data.
Example: int scores[10];

The third is the enumeration. An enum is a user-defined type made up of named integer constants. A variable of that type holds one value from the set, which replaces confusing numbers with names that are easy to read. It is the part of a program that names a set of choices.
Example: enum TrafficLight {RED, YELLOW, GREEN};

In short, a function is an action, an array is a list of data, and an enum is a set of names.

Conclusion

Structures, unions, and enumerations each solve a different problem in C. Structures bring different data types together into one organized unit. Unions save memory by letting several members share the same space. Enumerations replace confusing numbers with clear, readable names. Along with functions and arrays, they give a programmer the tools to write code that is well organized, efficient with memory, and simple to read and maintain.

sources

C Structure Explained: Syntax and Uses in Programming

Structures in C | C Programming | XQA Learn

C Structures Tutorial

What is use of structure in C?. In C programming, structures are a... | by TCCI - Tririd Computer Coaching Institute | Medium

C Structures - GeeksforGeeks

Structures in C Programming - Startertutorials

Unions in C Programming: Definition & Example - Lesson | Study.com

Datapro | What is a Union in C and why is it beneficial?

Unions in C - GeeksforGeeks

C Unions

Understanding Unions in C: Concept, Syntax, and use Cases

Can someone explain unions ? : r/C_Programming

Datapro | What is a Union in C and why is it beneficial?

Understanding Unions in C: Concept, Syntax, and use Cases

C Unions Explained: Memory Sharing, Declaration & Use Cases for Beginners

Enum: Definition, Purpose, and Examples

C Enum (Enumeration)

Understanding Enumeration in C: A Guide to Enums - Certbolt | IT Certification News

Enum Data Type in C: What it is and How to Use It | Simplilearn

Enumeration in C - Sanfoundry

Enum: Definition, Purpose, and Examples

C Enum (Enumeration)

C enumeration declarations | Microsoft Learn

https://cppreference.com/c/language/function_declaration

https://www.cs.uic.edu/~jbell/CourseNotes/C_Programming/Arrays.html